CSL7110 — Big Data

Assignment 4

Clustering, Web Search & PageRank

Roll Number: M25CSA033

Akanksha Kapil

Date: April 13, 2026

GitHub Repository: https://github.com/AkankshaKapil/MLBD_Assignment4

Part 1: Clustering

1.1 Understanding

This part implements two clustering algorithms on the UCI Spambase dataset: the Farthest-First Traversal (k-center) algorithm and K-Means++ seeding. The dataset consists of 4601 email feature vectors, each with 57 dimensions. The last column (class label) is excluded from clustering.

Farthest-First Traversal (k-Center)

The Farthest-First algorithm is a deterministic greedy approach to k-center clustering. It selects k centers by always picking the point that is farthest from all currently chosen centers.

Algorithm steps:

- Pick the first center as P[0] (assumption: deterministic starting point).
- Maintain min_dist[i] = squared distance from point P[i] to its nearest center.
- At each step, pick the point with the maximum min_dist as the next center.
- Update min_dist after each new center is added.
- Repeat until k centers are chosen.

Time complexity: O(|P| x k) — each of the k steps scans all points exactly once.

K-Means++ Seeding

K-Means++ is a probabilistic seeding algorithm that improves the initial center selection for K-Means clustering. It reduces the expected approximation error compared to random initialization.

Algorithm steps:

- Pick the first center uniformly at random from P.
- For each subsequent center, sample a point with probability proportional to its squared distance to the nearest existing center (D^2 weighting).
- Update min_dist after each new center is added.
- Repeat until k centers are chosen.

Time complexity: O(|P| x k) — each of the k steps scans all points exactly once.

1.2 Assumptions

- The last column of spambase.data is the class label and is excluded from feature vectors.
- For kcenter, the first center is always P[0] (deterministic choice).
- For kmeansPP, random.seed(42) is used for reproducibility.
- Vectors.sqdist() was replaced with a custom sqdist() helper using numpy due to its removal in newer PySpark versions. The result is mathematically identical.
- k = 10, k1 = 200 were used as the input integers (k < k1 as required).

1.3 Outputs

Experiment	Description	Result
Experiment 1	kcenter(P, k=10) — running time	0.0956 seconds, 10 centers
Experiment 2	kmeansPP(P, k=10) + kmeansObj(P, C)	Objective: 31250.987038
Experiment 3	kcenter(P, k1=200) -> kmeansPP(X, k=10) -> kmeansObj(P, C)	Objective: 30937.852274

```
============================================================
Experiment 1: kcenter(P, k=10)
  Running time: 0.0956 seconds
  Number of centers returned: 10
============================================================
Experiment 2: kmeansPP(P, k=10) + kmeansObj
  kmeansObj (kmeans++ centers): 31250.987038
============================================================
Experiment 3: kcenter(P, k1=200) → kmeansPP(X, k=10) → kmeansObj
  kmeansObj (coreset + kmeans++ centers): 30937.852274
============================================================

Observation: Experiment 3 should give an objective value comparable
to Experiment 2, since the k1-coreset provides good coverage before
applying kmeans++. Larger k1 = better coreset = lower objective.
```

Fig 1 : Part 1 output

1.4 Analysis

Experiment 3 produces a lower objective value (30937.85) than Experiment 2 (31250.99), validating the theoretical claim in the assignment: a larger k1 coreset extracted by kcenter provides better coverage of the data space, giving kmeansPP higher-quality candidates to seed from. With k1 = 200 spread-out coreset points, kmeansPP finds centers that minimize the average squared distance better than running on the full 4601-point dataset directly.

Part 2: Web Search — Inverted Index

2.1 Understanding

An inverted index is the core data structure behind search engines. For a collection of webpages, it maps each word to the set of pages and positions where that word appears. This enables fast lookup for search queries.

Given a collection C of webpages, the inverted index is defined as:

```
Inv(C) = { (w : {(p, k) : p in C, p[k] = w}) : w in W(C) }
```

where W(C) is the set of all non-stop words across all pages, and (p, k) indicates word w appears at position k in page p.

2.2 Classes Implemented

- MySet — set wrapper with union and intersection operations.
- Position — stores a (PageEntry, word_index) tuple.
- WordEntry — stores all positions across documents for a specific word.
- PageIndex — per-page index mapping each word to its positions.
- PageEntry — reads a webpage file, builds its PageIndex.
- MyHashTable — maps words to their global WordEntry using a hash function.
- InvertedPageIndex — global index over all pages using MyHashTable.
- SearchEngine — main stub; performs addPage and query actions.

2.3 Assumptions

- All words are converted to lowercase before indexing.
- Stop words are NOT stored in the index but their positions ARE counted when computing word indices.
- Punctuation characters are replaced with a space before tokenization.
- Plural/singular pairs treated as the same word: stacks->stack, structures->structure, applications->application. This list is treated as exhaustive.
- The connector words, punctuation, and plural mappings lists from the spec are treated as exhaustive — no additional exceptions are made.

2.4 Query Results

All 11 queries from actions.txt were executed and verified against answers.txt:

Action	Output	Expected	Pass
queryFindPagesWhichContainWord delhi	No webpage contains word delhi	No webpage contains word delhi	✓
queryFindPagesWhichContainWord stack	stack_datastructure_wiki	stack_datastructure_wiki	✓
queryFindPagesWhichContainWord wikipedia	stack_datastructure_wiki	stack_datastructure_wiki	✓
queryFindPositionsOfWordInAPage magazines stack_datastructure_wiki	Webpage ... does not contain word magazines	Webpage ... does not contain word magazines	✓
queryFindPagesWhichContainWord allain (before stack_cprogramming added)	No webpage contains word allain	No webpage contains word allain	✓
queryFindPagesWhichContainWord allain (after)	stack_cprogramming	stack_cprogramming	✓
queryFindPagesWhichContainWord C	stack_cprogramming	stack_cprogramming	✓
queryFindPagesWhichContainWord C++	stack_cprogramming	stack_cprogramming	✓
queryFindPagesWhichContainWord jdk	stack_oracle	stack_oracle	✓
queryFindPagesWhichContainWord function	stack_cprogramming, stack_datastructure_wiki, stackoverflow	stack_cprogramming, stack_datastructure_wiki, stackoverflow	✓
queryFindPagesWhichContainWord magazines	stackmagazine	stackmagazine	✓

Result: 11/11 queries passed (100% accuracy).

```
--- Running actions.txt ---

ACTION : queryFindPagesWhichContainWord delhi
OUTPUT : No webpage contains word delhi

ACTION : queryFindPagesWhichContainWord stack
OUTPUT : stack_datastructure_wiki

ACTION : queryFindPagesWhichContainWord wikipedia
OUTPUT : stack_datastructure_wiki

ACTION : queryFindPositionsOfWordInAPage magazines stack_datastructure_wiki
OUTPUT : Webpage stack_datastructure_wiki does not contain word magazines

ACTION : queryFindPagesWhichContainWord allain
OUTPUT : No webpage contains word allain

ACTION : queryFindPagesWhichContainWord allain
OUTPUT : stack_cprogramming

ACTION : queryFindPagesWhichContainWord C
OUTPUT : stack_cprogramming

ACTION : queryFindPagesWhichContainWord C++
OUTPUT : stack_cprogramming

ACTION : queryFindPagesWhichContainWord jdk
OUTPUT : stack_oracle

ACTION : queryFindPagesWhichContainWord function
OUTPUT : stack_cprogramming, stack_datastructure_wiki, stackoverflow

ACTION : queryFindPagesWhichContainWord magazines
OUTPUT : stackmagazine
```

Figure 2: Part 2 output

```
--- Comparing with answers.txt ---

[✓ PASS] Expected: "No webpage contains word delhi"
[✓ PASS] Expected: "stack_datastructure_wiki"
[✓ PASS] Expected: "stack_datastructure_wiki"
[✓ PASS] Expected: "Webpage stack_datastructure_wiki does not contain word magazines"
[✓ PASS] Expected: "No webpage contains word allain"
[✓ PASS] Expected: "stack_cprogramming"
[✓ PASS] Expected: "stack_cprogramming"
[✓ PASS] Expected: "stack_cprogramming"
[✓ PASS] Expected: "stack_oracle"
[✓ PASS] Expected: "stack_cprogramming, stack_datastructure_wiki, stackoverflow"
[✓ PASS] Expected: "stackmagazine"

All outputs match answers.txt!
```

Figure 3: Part 2 output

Part 3: PageRank on Spark

3.1 Understanding

PageRank is an algorithm originally developed by Google to rank web pages by importance. It models a random surfer who follows links randomly, with a probability (1 - beta) of teleporting to a random page.

Given a directed graph G = (V, E) with n nodes, the PageRank vector r satisfies:

```
r = ((1 - beta) / n) * A + beta * M * r
```

where A is the n x 1 unit vector, beta = 0.8 is the damping factor, and M is the column-stochastic transition matrix where M[i][j] = 1/deg(j) if edge j->i exists.

3.2 Iterative Algorithm

The PageRank vector is computed iteratively:

- Initialize r0 = (1/n) * A — all nodes start with equal rank.
- For each iteration: r(i) = ((1-beta)/n) * A + beta * M * r(i-1)
- Repeat for 40 iterations as specified.

In Spark, M is represented as an RDD of (src, dst, weight) triples. At each iteration, contributions are computed by joining the edges RDD with the current ranks RDD, then summing by destination node.

3.3 Assumptions

- Multiple directed edges between the same pair of nodes are treated as a single edge (deduplicated using .distinct()).
- beta = 0.8 and iterations = 40 as specified.
- The graph dataset was provided externally (the original GitHub link in the assignment returned empty files). The dataset used has 100 nodes in small.txt and 1000 nodes in whole.txt.
- Nodes that appear only as destinations (no outgoing edges) receive the teleportation value as their rank contribution.

3.4 Validation on small.txt

Metric	Value
Nodes loaded	100
Top node (Node 53)	0.035731
Expected top score	~0.036
Validation	PASSED — top score matches expected value

```
Running PageRank on small.txt (53 nodes)...

  Graph loaded: 100 nodes
[Stage 125:====================================================>(241 + 3) / 244]
Top node score: 0.0357  (expected ≈ 0.036)

Top 5 nodes (small graph):
  Node   53 : 0.035731
  Node   14 : 0.034171
  Node   40 : 0.033630
  Node    1 : 0.030006
  Node   27 : 0.029720
```

Figure 4: Part 3 53 node output

3.5 Results on whole.txt

Graph: 1000 nodes, 8192 edges, beta = 0.8, 40 iterations.

Rank	Node ID	PageRank Score
Top 1	263	0.00202029
Top 2	537	0.00194334
Top 3	965	0.00192545
Top 4	243	0.00185263
Top 5	285	0.00182737
Bottom 1	408	0.00038780
Bottom 2	424	0.00035482
Bottom 3	62	0.00035315
Bottom 4	93	0.00035136
Bottom 5	558	0.00032860

```
Running PageRank on whole.txt (1000 nodes, 40 iterations, β=0.8)...
  Graph loaded: 1000 nodes
[Stage 251:==================================================> (239 + 4) / 244]
--- Top 5 nodes (highest PageRank) ---
  Node   263 : 0.00202029
  Node   537 : 0.00194334
  Node   965 : 0.00192545
  Node   243 : 0.00185263
  Node   285 : 0.00182737

--- Bottom 5 nodes (lowest PageRank) ---
  Node   408 : 0.00038780
  Node   424 : 0.00035482
  Node    62 : 0.00035315
  Node    93 : 0.00035136
  Node   558 : 0.00032860
```

Figure 5 : Part 3 output 1000 nodes

3.6 Analysis

The PageRank scores show a clear spread — the top node (263) scores approximately 6x higher than the bottom node (558). This is expected given the directed cycle structure of the graph ensures connectivity, while additional random edges create hubs with higher in-degree that accumulate more rank. The convergence after 40 iterations with beta = 0.8 produces a stable distribution consistent with the teleportation model.

Summary

Part	Task	Status	Key Result
Part 1	Clustering (kcenter + kmeansPP)	Complete	Exp3 obj (30937.85) < Exp2 obj (31250.99)
Part 2	Inverted Index Search Engine	Complete	11/11 queries correct
Part 3	PageRank on Spark	Complete	Top node: 263 (score: 0.00202)

GitHub Repository: https://github.com/AkankshaKapil/MLBD_Assignment4